

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 6

Student Name: K. Saranya

Roll No.: A24126552090

Date: 30-08-2026

1. TITLE

Implement Modules in Node.js.

2. AIM

To understand and implement built-in, user-defined, and third-party modules in Node.js, and demonstrate their usage through file handling, operating system information, path manipulation, HTTP server creation, URL parsing, events, and Express.js.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept of modules in Node.js.
- To use built-in Node.js modules.
- To perform file operations using the fs module.
- To obtain operating system information using the os module.
- To manipulate file paths using the path module.
- To create a basic HTTP server using the http module.
- To parse URLs using the url module.
- To create and handle custom events using the events module.
- To understand user-defined modules using module.exports.
- To import and use functions from a user-defined module.
- To understand the use of the Express.js module for creating a web server.

4. THEORY

Node.js provides a modular architecture that allows an application to be divided into smaller, reusable components called modules. Modules help organise code and make applications easier to maintain and reuse.

Node.js modules can be broadly classified into three types: Built-in Modules, User-defined Modules, and Third-party Modules.

Built-in Modules

Node.js provides several built-in modules that can be directly imported using the `require()` function. The following built-in modules are demonstrated in this experiment:

- fs — used for file system operations such as creating and writing files.
- os — provides information about the operating system and CPU.
- path — provides utilities for working with file and directory paths.
- http — used to create HTTP servers.
- url — used to parse and work with URLs.
- events — provides the EventEmitter class for creating and handling custom events.

User-defined Module

A user-defined module is a JavaScript file created by the developer. Functions or variables can be exported using `module.exports` and imported into another file using `require()`. In this experiment, arithmetic functions — addition, subtraction, multiplication, and division — are exported from a user-defined module.

Express Module

Express.js is a third-party web framework for Node.js that simplifies the process of creating web servers and handling HTTP requests and responses. In the Express program, a GET route is created for `/`, which sends the message “Hello from Express!” to the browser.

The basic module working flow is:

Module Creation / Built-in Module → require() → Functionality → Output

5. ALGORITHM / PROCEDURE

1. Create a new Node.js project.
2. Initialize the project using `npm init`.
3. Import the required built-in modules using `require()`.
4. Use the `fs` module to create and write data into a text file.
5. Use the `os` module to display the operating system platform and CPU core count.
6. Use the `path` module to obtain the base name of a file path.
7. Use the `http` module to create a basic Node.js HTTP server.
8. Start the HTTP server on port 3000.
9. Use the `url` module to create and parse a URL.
10. Display the protocol, host, path, and query parameters.
11. Import the `EventEmitter` class from the `events` module.
12. Create a custom event using `EventEmitter`.
13. Register an event listener and trigger the event.
14. Create a user-defined module containing arithmetic functions.
15. Export the functions using `module.exports`.
16. Import the user-defined module using `require()`.
17. Use the exported functions in another JavaScript file.
18. Install Express.js using `npm install express`.
19. Create an Express application.
20. Define a GET route and start the Express server.
21. Execute the programs and verify the output.

6. PROGRAM

Program 1: Built-in Modules — *builtIn.js*

```
builtIn.js
1  //fs module
2  const fs = require("fs");
3  fs.writeFileSync("hello.txt", "hello node.js");
4  console.log("File created");
5
6  //os module
7  const os = require("os");
8  console.log(os.platform());      //os which we are using
9  console.log(os.cpus().length);   //no. of cpu cores
10
11 //path module
12 const path = require("path");
13 console.log(path.basename("C:/Users/SARANYA/Desktop/A24126552090/week6/hello.txt"));
14
15 // http module
16 const http = require("http");
17 const server = http.createServer((req, res) => {
18     res.end("Hello from Node.js!");
19 });
20 server.listen(3000);
21 console.log("Server running on port 3000");
22
23 // url module
24 const { URL } = require("url");
25 const myURL = new URL("https://example.com:8080/products?id=101");
26 console.log("Protocol:", myURL.protocol);
27 console.log("Host:", myURL.host);
28 console.log("Path:", myURL.pathname);
29 console.log("Query:", myURL.search);
30
31 //events module
32 const EventEmitter = require("events");
33 const myEvent = new EventEmitter();
34 myEvent.on("welcome", () => {
35     console.log("Welcome to Node.js!");
36 });
37 myEvent.emit("welcome");
```

Program 2: Express Module — *module.js*

```
module.js
1  const express = require("express");
2  const app = express();
3
4  app.get("/", (req, res) => {
5      res.send("Hello from Express!");
6  });
7
8  app.listen(3000, () => {
9      console.log("Server running on port 3000");
10 });
```

Program 3: User-defined Module — *app.js*

```
app.js
```

```

1 // exporting user defined module
2 function add(a, b) {
3     return a + b;
4 }
5 function sub(a, b) {
6     return a - b;
7 }
8 function mul(a, b) {
9     return a * b;
10 }
11 function div(a, b) {
12     return a / b;
13 }
14
15 module.exports = {
16     add, sub, mul, div
17 };

```

Program 3: User-defined Module — user.js

```

user.js
1 const math = require("./app");
2
3 console.log("Add:", math.add(10, 5));
4 console.log("Sub:", math.sub(10, 5));
5 console.log("Mul", math.mul(10, 5));
6 console.log("Div", math.div(10, 5));

```

Supporting File: hello.txt

The fs module creates the file hello.txt and writes the following content into it: hello node.js

7. CODE EXPLANATION

Code / Concept	Explanation
require("fs")	Imports the built-in File System module.
writeFileSync()	Creates or overwrites a file and writes data synchronously.
hello.txt	Text file created by the fs module.
require("os")	Imports the Operating System module.
os.platform()	Returns information about the operating system platform.
os.cpus()	Returns information about the available CPU cores.
require("path")	Imports the Path module.
path.basename()	Returns the final portion of a file path.
require("http")	Imports the HTTP module.
http.createServer()	Creates an HTTP server.
res.end()	Sends a response and ends the HTTP response.
server.listen(3000)	Starts the HTTP server on port 3000.
require("url")	Imports URL-related functionality.
new URL()	Creates a URL object for parsing URL components.

Code / Concept	Explanation
protocol / host / pathname / search	Return the protocol, hostname and port, path, and query string of the URL.
require("events")	Imports the Node.js Events module.
EventEmitter	Used to create and manage custom events.
.on() / .emit()	Registers an event listener and triggers an event.
require("express")	Imports the Express.js framework.
express() / app.get() / app.listen()	Creates an Express application, defines a GET route, and starts the server.
module.exports	Exports functions or objects from a user-defined module.
add() / sub() / mul() / div()	Return the sum, difference, product, and quotient of two numbers.

8. TEST CASES AND EXECUTION DATA

The Node.js programs were executed in the terminal and the functionality of the built-in, user-defined, and Express modules was verified against the output shown below.

Test Case: TC-01
Action: Execute node builtin.js in terminal
Expected Result: fs.writeFileSync() creates hello.txt and logs "File created".
Actual Result: "File created" printed in terminal and hello.txt was generated.
Status: Pass

```
PS C:\Users\SARANYA\OneDrive\Documents\fullstackwebdevelopment\02-records\week-6-modules\code> node builtin.js
File created
```

Test Case: TC-02
Action: Verify System & Path information via os and path modules in builtin.js
Expected Result: Display OS platform (win32), CPU cores count (16), and base file name (hello.txt).
Actual Result: Correct OS platform, core count, and basename rendered cleanly.
Status: Pass

```
PS C:\Users\SARANYA\OneDrive\Documents\fullstackwebdevelopment\02-records\week-6-modules\code> node builtin.js
File created
win32
16
hello.txt
```

Test Case: TC-03
Action: Execute URL parsing and custom Event emission in builtin.js
Expected Result: URL components (protocol, host, path, query) and EventEmitter welcome message display correctly.
Actual Result: URL parameters parsed accurately and "Welcome to Node.js!" printed on event trigger.
Status: Pass

```
Protocol: https:
Host: example.com:8080
Path: /products
Query: ?id=101
Welcome to Node.js!
█
```

Test Case: TC-04
Action: Execute node module.js (Express Server)
Expected Result: Express framework initializes and listens on port 3000.
Actual Result: "Server running on port 3000" logged in terminal.
Status: Pass

```
C:\Users\SARANYA\OneDrive\Documents\fullstackwebdevelopment\02-records\week-6-modules\code>node module.js
Server running on port 3000
```

Test Case: TC-05
Action: Execute node user.js (User-defined Math Module)
Expected Result: Imported arithmetic functions (add, sub, mul, div) execute and return accurate values.
Actual Result: Calculated outputs displayed correctly (Add: 15, Sub: 5, Mul: 50, Div: 2).
Status: Pass

```
C:\Users\SARANYA\OneDrive\Documents\fullstackwebdevelopment\02-records\week-6-modules\code>node user.js
Add: 15
Sub: 5
Mul 50
Div 2
```

9.OUTPUT

Output: Node.js Modules Execution

The following screenshot shows the terminal execution of the Node.js modules programs. It includes the outputs generated by the built-in modules such as fs, os, path, http, url, and events (builtin.js), the module server (module.js), and the exported arithmetic functions (user.js).

```
C:\Users\SARANYA\OneDrive\Documents\fullstackwebdevelopment\records\week-6>node builtin.js
File created
win32
16
hello.txt
Server running on port 3000
Protocol: https:
Host: example.com:8080
Path: /products
Query: ?id=101
Welcome to Node.js!
^C
C:\Users\SARANYA\OneDrive\Documents\fullstackwebdevelopment\records\week-6>node module.js
Server running on port 3000
^C
C:\Users\SARANYA\OneDrive\Documents\fullstackwebdevelopment\records\week-6>node user.js
Add: 15
Sub: 5
Mul 50
Div 2
C:\Users\SARANYA\OneDrive\Documents\fullstackwebdevelopment\records\week-6>
```

10.RESULT

Thus, the different types of Node.js modules were successfully implemented and executed. Built-in modules such as fs, os, path, http, url, and events were used successfully. A user-defined module was created using module.exports, and the Express.js module was used to create a basic web server. The programs were executed successfully and the expected results were obtained.